

DEEP LEARNING

Deep Neural Networks Part II

Subhankar Roy

Department of Management, Information and Production Engineering (DIGIP)
University of Bergamo

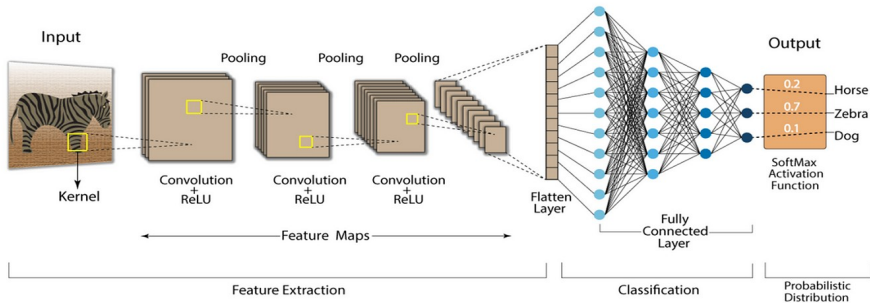
Table of Contents

1. Convolutional Neural Network

2. Transfer Learning

- 2.1 Fine-tuning Procedures
- 2.2 Fine-tuning in Practice
- 2.3 Applications

Recap: Convolutional Neural Network

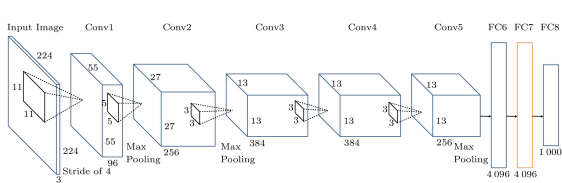


CNNs can be divided into the following parts:

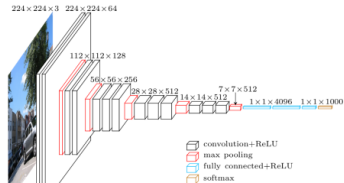
- **Feature Extractor:** Repeating patterns of Convolution → ReLU → Pooling blocks.
- **Classifier:** A flattening operation, followed by fully connected layers. The output layer contains as many neurons as the number of classes we want to predict.
- **Output Probability Distribution:** A softmax function is applied to the output logits to get a probability distribution over the classes.

Training CNNs

- **Training** CNNs mean **learning the parameters** of:
 - ▶ Feature extractor, i.e., the convolution layers.
 - ▶ Classifier, i.e., the fully connected layers.
 - ▶ Note that pooling and ReLU has no learnable parameters.



Alexnet (60 M parameters)



VGG16 (138 M parameters)

- Also note that in the above two networks, there are many parameters to learn!

Classifying Popular Animals



- Big networks need lots of training data (the more the better).
- When we are dealing with classifying popular animals (e.g., cats, dogs, elephants), we can find a lot of images on the internet.
- Training these big models from **scratch** (i.e., starting from randomly initialized weights) is feasible.

Classifying Rare Animals



Porpoise



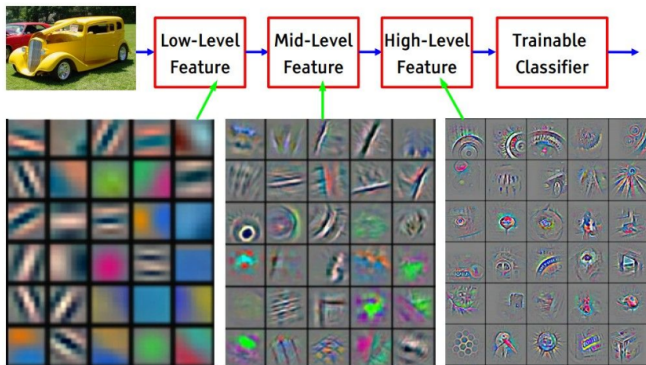
Saola



Bald Ibis

- When dealing with rare animals (or rarely photographed animals), the data is not enough to train the big networks from scratch!
- Training with limited data can cause severe **overfitting** (too many parameters → too complex function).
- Q: How can we classify images when we have limited training data?

Visualizing What CNNs Learn



- CNNs learn filters that detect specific features at different layers:
 - ▶ Early layers → edges, corners, textures
 - ▶ Middle layers → patterns, shapes
 - ▶ Deep layers → object parts, abstract concepts

Reusing Existing Knowledge

- The early layers of the CNN learn **generic low-level features** (e.g., edges, textures, corners) that are common across datasets.
- Only the later layers (or deep layers) learn **task-specific high-level features** (e.g., animal parts, object shapes).
- **Key idea:** We don't have re-learn everything from scratch. Rather, we will reuse the foundation (low-level features), and **re-learn** only the later layers for our new task (or dataset).
- This is the key idea behind what we call **transfer learning**.

Transfer Learning

- Definition on Wikipedia:

*“Transfer learning (TL) is a technique in machine learning (ML) in which knowledge learned from a task is **re-used** in order to boost performance on a related task.”*

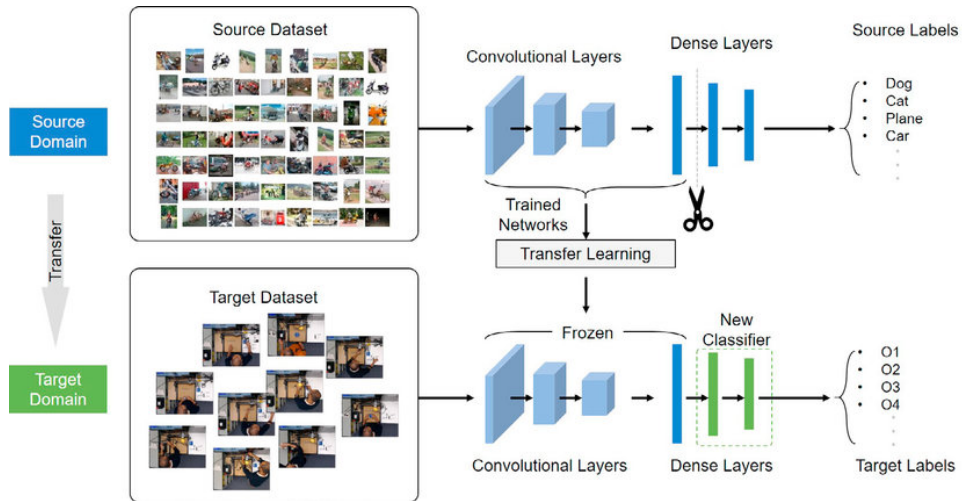
- **Reusing knowledge:** For example, knowledge gained while learning to recognize cars could be applied when trying to recognize trucks.
- Especially useful, when we have **limited data** and **limited compute resources**.
- Somewhat similar to how we **humans learn**. If you know how to skate on ice, then it is straightforward to learn to snowboard!

Transfer Learning: Basics

- Train a big model on a large “source” dataset on a given downstream task (e.g., classification of popular animals). Once trained, save the parameters. This is also called **pre-trained model**.
- Use the parameters of the pre-trained model as a starting point (recall weight initialization) for a smaller “target” dataset that is derived from a new downstream task (e.g., classification of rare animals, or medical images).
- Update or “fine-tune” only a small part of the big model (e.g., fully connected or dense layers) on the target dataset, while “freezing”¹ most of the parameters of the big model.
- Intuition being, learn what is new while reusing the common patterns.

¹freezing means preventing specific parameters from being updated

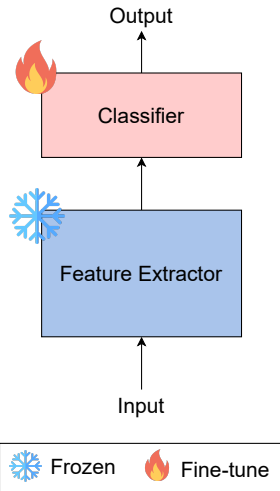
Transfer Learning: Schema



Fine-tuning Procedure: Type 1

Given a new target dataset:

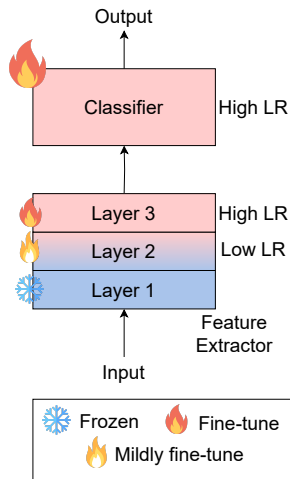
- We retain (or keep) the feature extractor of the pre-trained model.
- We discard the classifier of the pre-trained model, and initialize a new classifier with random weights.
- We **freeze** the weights of the feature extractor, and **fine-tune** (or learn from scratch) only the classifier.
- Fine-tuning is nothing but updating the weights using SGD.



Fine-tuning Procedure: Type 2

Given a new target dataset:

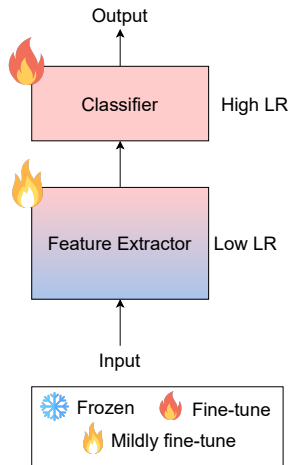
- We initialize the weights, the same as above.
- Instead of completely freezing the feature extractor, we **differentially fine-tune** each layer of the feature extractor.
- Earlier layers are completely frozen.
- Middle layers are mildly fine-tuned using a low learning rate (LR), e.g., 0.001.
- Deeper layers and the classifier are fine-tuned with a high LR, e.g., 0.1.
- Idea is to not to change the low-level features too drastically.



Fine-tuning Procedure: Type 3

Given a new target dataset:

- In this case we fine-tune both the feature extractor and the classifier.
- Higher LR is used for the classifier, but lower LR is used for the feature extractor.



Fine-tuning in Practice

- There are a couple of design choices to be made during transfer learning:
 1. Which layers to fine-tune and which layers to keep frozen?
 2. If we are fine-tuning a layer, how strongly do we want to fine-tune (i.e., learning rate value)?
- The answers will depend on a few things:
 1. How similar are the source and the target datasets?
 2. How many images are in the target dataset?
 3. How big is the pre-trained model?
- Things to keep in mind:
 1. Too aggressive fine-tuning will ruin the previously learned knowledge in the pre-trained model.
 2. Too conservative fine-tuning will not be sufficient to learn something useful.
 3. There is a trade-off between the two.

Fine-tuning in Practice: Similarity between Datasets



- If the target dataset is **very similar** to the source dataset then we opt for the Type 1 fine-tuning.
- If they are **somewhat similar** (or dissimilar) then we opt for Type 2.
- When they are **very dissimilar** then we opt for Type 3.

Fine-tuning in Practice: Number of Training Images

- The choice will also depend on the **number of images** in the target dataset.
- If we have a very **few images** (e.g., 3 images per category) in the target dataset, then we should try to fine-tune as less as possible, i.e., Type 1.
- If we have a **lot of images** (e.g., several hundreds per category) in the target dataset, then we can afford to fine-tune more, i.e., Type 3.
- For everything in between we can use Type 2.

Fine-tuning in Practice: Size of the Pre-trained Model

- The choice of fine-tuning depends on the [size](#) of the pre-trained model.
- Very big models have generally learned a good representation from a large source dataset. This means that we do not need to fine-tune a lot – Type 1 or Type 2.
- Big models also have a lot of parameters, so we do not want to fine-tune a lot if we do not have a lot of images in the target dataset → overfitting.
- If the pre-trained model is reasonably sized (i.e., not too big) then we can opt for Type 3.

Transfer Learning Tips

- There is no clear recipe on what is the best approach, and it depends on several things.
- A general rule of thumb is: start from Type 1 and slowly work towards Type 3 (by monitoring the validation accuracy).
- Other considerations: [time](#) and [computational resources](#).
- Type 1 is both time and compute friendly, whereas Type 3 is the opposite.

Applications of Transfer Learning

- Learn from **simulations** (self-driving cars), i.e., pre-train on simulated data (cheap to collect) and fine-tune on real-world driving data (expensive to collect).
- **Domain adaptation**, i.e., pre-train on driving data collected on sunny days (frequently occurring), and fine-tune on data collected on foggy days (rarely occurring).
- The idea of transfer learning goes beyond the image data and CNNs. For instance, pre-train a **language model** on high-resource language (e.g., English or Spanish, since there are plenty of people speaking these languages), and transfer this knowledge to learn resource-poor language (e.g., Nepalese).
- Transfer learning is like a *Swiss-army knife*, has a wide range of real-world applications.

References and Links

- For additional reading materials refer to the following links
 - ▶ “[CS231n Lecture Notes](#)”², Module 2.

²<https://cs231n.github.io/>